



北京邮电大学
Beijing University of Posts and Telecommunications

BUPT Compilers Lab4 2025Fall

Name:

Student ID:

Class Number:

Container Number:

1 Code

1.1 lex.l

Add a LEADINGZERO type to help the syntax.

```
%{
    #include "syntax.tab.h"
}%
%option noyywrap

unic u[0-9a-fA-F]{4}
esc  \\([\\\\"/bfnrt]|{unic})
scp  [^"\\\x00-\x1f]
string \"({esc}|{scp})*\"

int  0|[1-9][0-9]*
frac \.[0-9]+
exp  [Ee][+-]?[0-9]+
number -?{int}{frac}?{exp}?
leadingzero 0{int}

empty [ \n\r\t]

%%

"{" { return LC; }
"}" { return RC; }
"[" { return LB; }
"]" { return RB; }
":" { return COLON; }
"," { return COMMA; }

"true" { return TRUE; }
"false" { return FALSE; }
"null" { return VNULL; }

{string} { return STRING; }
{number} { return NUMBER; }
{leadingzero} {return LEADINGZERO;}

{empty} {}

. { printf("lexical error: %s\n", yytext); }
```

1.2 syntax.y

```

%{
    #include "lex.yy.c"
    void yyerror(const char*);
%}

%token LC RC LB RB COLON COMMA
%token STRING NUMBER LEADINGZERO
%token TRUE FALSE VNULL
%%

Json:
    Value
    | Value COMMA error { printf("Comma after the close,
recovered"); }
    | Value RB error { printf("Extra close, recovered"); }
    ;
Value:
    Object
    | Array
    | STRING
    | NUMBER
    | TRUE
    | FALSE
    | VNULL
    ;
Object:
    LC RC
    | LC Members RC
    | Object Value error { puts("Extra value after close,
recovered"); }
    | LC Values RC error { puts("Comma instead of colon,
recovered"); }
    | LC Member COMMA error { puts("Comma instead if closing brace,
recovered"); }
    ;
Members:
    Member
    | Members COMMA Member
    | Members COMMA error { puts("Extra comma, recovered"); }
    ;
Member:
    STRING COLON Value
    | STRING COLON error Value { puts("Double colon, recovered"); }

```

```

    | STRING Value error { puts("Missing colon, recovered"); }
    | STRING COLON LEADINGZERO error { puts("Numbers cannot have
leading zeroes, recovered"); }
    ;
Array:
    LB RB
    | LB Values RB
    | LB Values RC error { puts("mismatch, recovered"); }
    | LB Member RB error { puts("Colon instead of comma,
recovered"); }
    | LB Values error { puts("Unclosed array, recovered"); }
    ;
Values:
    Value
    | Values COMMA Value
    | Values COMMA error { puts("extra comma, recovered"); }
    | Values COMMA COMMA error { puts("double extra comma,
recovered"); }
    | COMMA Values error { puts("<-- missing value, recovered"); }
    ;
%%

void yyerror(const char *s){
    printf("syntax error: ");
}

int main(int argc, char **argv){
    if(argc != 2) {
        fprintf(stderr, "Usage: %s <file_path>\n", argv[0]);
        exit(-1);
    }
    else if(!(yyin = fopen(argv[1], "r"))) {
        perror(argv[1]);
        exit(-1);
    }
    yyparse();
    return 0;
}

```

2 Run

The commands were used:

```
make jp
python3 jsonparser_test.py
```

The screenshot of the compilation and execution results.

```
root@53e88d23dad3:/mnt/Workspace/lab4/jp# make jp
flex lex.l
bison -t -d syntax.y
gcc syntax.tab.c -lfl -ly -o jp.out

root@53e88d23dad3:/mnt/Workspace/lab4/jp# python3 jsonparser_test.py
For file fail02.json:
-----
["Unclosed array"]
-----
syntax error: Unclosed array, recovered
#####
For file fail04.json:
-----
["extra comma",]
-----
syntax error: extra comma, recovered
#####
For file fail05.json:
-----
["double extra comma",,,]
-----
syntax error: double extra comma, recovered
#####
For file fail06.json:
-----
[ , "<-- missing value"]
-----
syntax error: <-- missing value, recovered
#####
For file fail07.json:
-----
["Comma after the close"],
-----
syntax error: Comma after the close, recovered
#####
For file fail08.json:
-----
["Extra close"]]
-----
syntax error: Extra close, recovered
#####
For file fail09.json:
-----
{"Extra comma": true,}
-----
syntax error: Extra comma, recovered
#####
```

```

For file fail10.json:
-----
{"Extra value after close": true} "misplaced quoted value"
-----
syntax error: Extra value after close, recovered
#####
For file fail13.json:
-----
{"Numbers cannot have leading zeroes": 013}
-----
syntax error: Numbers cannot have leading zeroes, recovered
#####
For file fail19.json:
-----
{"Missing colon" null}
-----
syntax error: Missing colon, recovered
#####
For file fail20.json:
-----
{"Double colon": null}
-----
syntax error: Double colon, recovered
#####
For file fail21.json:
-----
{"Comma instead of colon", null}
-----
syntax error: Comma instead of colon, recovered
#####
For file fail22.json:
-----
["Colon instead of comma": false]
-----
syntax error: Colon instead of comma, recovered
#####
For file fail32.json:
-----
{"Comma instead if closing brace": true,
syntax error: Comma instead if closing brace, recovered
#####

For file fail33.json:
-----
["mismatch"}
-----
syntax error: mismatch, recovered
#####
Recovered/Total: 15/15
root@53e88d23dad3:/mnt/Workspace/lab4/jp#

```

The syntax has no conflict and it passed all the 15 tests.